

Phase 4 Assignment 4B - Backend API Projects

Goal: Master Phase 1 through Phase 4 by building production-shaped, real-time, secure backend systems that combine databases, authentication, background processing, caching, and live communication. Scope: Only use topics taught in Lessons 1-39. Standard: A major capstone. These projects are substantially harder than the Phase 1, 2, and 3 assignments. They are real-time, multi-user, authenticated systems, not CRUD apps.

Allowed Scope

Use only concepts from Phase 1, Phase 2, Phase 3, and Phase 4:

Phase 1 - Fundamentals - FastAPI app setup, routing, path/query/body parameters - Pydantic models, nested models, `Field()`, status codes, `HTTPException`, `JSONResponse`, `Response`

Phase 2 - Core Features - `response_model`, separate input/output/update schemas, `field_validator`, `model_validator`, `model_config` - Custom exception handlers, `Depends()` (sub-dependencies, class-based, `yield`), middleware, `APIRouter` - Form data and file uploads, headers and cookies

Phase 3 - Database Integration - SQLAlchemy 2.0 or `SQLModel` models, relationships (one-to-many, many-to-many, self-referential, association objects) - Database `Session/AsyncSession` as a dependency, full CRUD, `from_attributes` + schema/model separation - Alembic migrations, async SQLAlchemy, database transactions, Redis and MongoDB (optional per project)

Phase 4 - Advanced Features - Async vs sync reasoning, `run_in_threadpool` - Authentication (password hashing with `bcrypt`, OAuth2 Password Flow, JWT access/refresh tokens), authorization (RBAC), API keys - Background tasks (`BackgroundTasks`) - WebSockets (connection manager, broadcasting) - Server-Sent Events and `StreamingResponse` - CORS (`CORSMiddleware`) - Rate limiting (`slowapi`, per-IP / per-user) - Caching (in-memory, Redis, `fastapi-cache2`) - Pagination (limit/offset and cursor-based) - Filtering, sorting, searching (whitelisted, injection-safe) - Internationalization (optional), GraphQL via Strawberry (optional)

Do not use:

- Automated testing frameworks (`pytest`, `coverage`) as a required deliverable - Phase 5
- Structured logging / observability stacks (`structlog`, `Sentry`, `Prometheus`) - Phase 6
- `pydantic-settings`-based multi-environment configuration systems - Phase 6
- Docker, deployment, CI/CD, production servers (Gunicorn), reverse proxies - Phase 6
- Celery / RQ / ARQ or external job queues - use the built-in `BackgroundTasks`; you may discuss where a queue would be better, but do not implement one
- API versioning schemes, custom OpenAPI overrides - Phase 6
- Any Phase 5+ feature, even if it would be useful in a real system

Important: These are multi-user systems, so **real authentication is now required**. Replace the `X-User-Id`-style header stand-ins from earlier assignments with genuine JWT-based auth and role-based authorization. Device or service callers should authenticate with API keys.

Global Rules

- Build backend APIs only. Do not build a frontend (a minimal HTML page to exercise a WebSocket or SSE stream is acceptable as a test harness, not as a product).
- Do not provide solutions, code, pseudocode, or implementation hints in your submission notes.
- **Persist all durable data in a real relational database** using SQLAlchemy 2.0 or SQLModel, with **Alembic migrations** for the schema.
- **Implement real authentication and authorization**: hashed passwords, JWT access + refresh tokens, an OAuth2 password-flow login, a `get_current_user` dependency, and role-based access control. Use API keys for non-human (device/service) callers where specified.
- **Include real-time communication** (WebSockets and/or SSE) where the project requires it, using a connection manager and broadcasting.
- **Use background tasks** for work that should happen after the response (notifications, receipts, dispatch, aggregation), using the built-in `BackgroundTasks`.
- **Apply rate limiting** to sensitive or expensive endpoints (login, writes, ingestion) and return 429 correctly.
- **Apply caching** (in-memory or Redis) to hot read paths, with correct TTL and invalidation on writes.
- Use **cursor-based pagination** for large or fast-growing collections; limit/offset is acceptable for small admin lists.
- Enforce integrity at the database level (unique, foreign key, not-null) in addition to Pydantic validation.
- Configure **CORS** for a hypothetical web client origin.
- Split code into modules and routers: separate database setup, models, schemas, dependencies, auth, routers, real-time managers, and background tasks.
- Each project must run with `uvicorn main:app --reload` after `alembic upgrade head`.
- Design every project so that another developer could safely extend it.

Recommended persistence and infrastructure pattern:

- One database per project (SQLite is fine locally; the code should target PostgreSQL by changing only the connection URL).
- An `alembic/` directory with real, ordered migration scripts.
- Redis for caching and/or rate-limit storage where a shared store is needed (an in-memory fallback is acceptable for local single-process runs, but note the limitation).
- Server-generated primary keys; never trust a client-supplied id.
- Output schemas that hide internal fields (password hashes, raw API keys, internal notes).

Recommended evaluation mindset:

- Correct, secure authentication and authorization
- Correct real-time design (connection lifecycle, broadcasting, disconnect handling)
- Safe concurrency around shared state (atomic transactions where money, bids, or dispatch are involved)
- Sensible caching, rate limiting, and pagination
- Clean modular architecture and consistent error handling

Project 1 - Real-Time Team Chat and Collaboration API

Difficulty Level

Advanced

Estimated Completion Time

24-32 hours

Project Overview

Build the backend for a team chat platform (in the spirit of Slack or Discord) where users join workspaces, talk in channels, thread replies, react to messages, and see each other's presence in real time. This project combines a relational data model with JWT authentication, workspace-scoped authorization, WebSocket broadcasting, background notifications, cursor-paginated history, and rate-limited message sending.

Problem Statement

A company needs a backend to:

- Register and authenticate users
- Let users create workspaces and invite others
- Organize conversations into public and private channels
- Deliver messages to channel members in real time
- Support threaded replies and emoji reactions
- Show who is online and who is typing
- Search and page back through message history

The system must behave like a real chat backend where authorization is workspace- and channel-scoped and delivery is instant.

Functional Requirements

- Register, log in, refresh tokens, and log out
- Create and manage workspaces; invite and remove members
- Assign workspace roles (owner, admin, member)
- Create public and private channels and manage channel membership
- Send, edit, and delete messages, including threaded replies
- Add and remove emoji reactions
- Broadcast new messages, edits, reactions, presence, and typing indicators over WebSockets
- Retrieve paginated channel history and search messages

Non-Functional Requirements

- Real authentication with hashed passwords and JWT access/refresh tokens
- WebSocket connection manager with per-workspace broadcasting and clean disconnect handling
- Cursor-based pagination for message history
- Rate limiting on message sending
- Caching of channel membership and workspace metadata
- CORS configured for a web client origin

API Requirements

- Use routers such as `auth`, `workspaces`, `channels`, `messages`, `reactions`, and a `ws` real-time module
- Use JWT for human users; protect every non-auth route with a `get_current_user` dependency
- Enforce workspace and channel membership with dependencies before any read or write
- Read message history with a cursor pagination scheme (newest-first, stable under new inserts)
- Broadcast events to connected members through a connection manager keyed by workspace
- Rate limit message creation per user

Required API Endpoints

Method	Path	Purpose
POST	<code>/auth/register</code>	Create a user
POST	<code>/auth/login</code>	OAuth2 password login, returns tokens
POST	<code>/auth/refresh</code>	Exchange a refresh token for a new access token
GET	<code>/auth/me</code>	Current user profile
POST	<code>/workspaces</code>	Create a workspace
GET	<code>/workspaces</code>	List the caller's workspaces
POST	<code>/workspaces/{workspace_id}/invites</code>	Invite a user by email
POST	<code>/invites/{token}/accept</code>	Accept an invite
GET	<code>/workspaces/{workspace_id}/members</code>	List members
PATCH	<code>/workspaces/{workspace_id}/members/{user_id}</code>	Change a member role
DELETE	<code>/workspaces/{workspace_id}/members/{user_id}</code>	Remove a member
POST	<code>/workspaces/{workspace_id}/channels</code>	Create a channel

GET	/workspaces/{workspace_id}/channels	List channels the caller can see
POST	/channels/{channel_id}/members	Add a member to a private channel
POST	/channels/{channel_id}/messages	Send a message
GET	/channels/{channel_id}/messages	Cursor-paginated history
PATCH	/messages/{message_id}	Edit a message
DELETE	/messages/{message_id}	Delete a message
POST	/messages/{message_id}/reactions	Add a reaction
DELETE	/messages/{message_id}/reactions/{emoji}	Remove a reaction
GET	/channels/{channel_id}/search	Search messages
WS	/ws/workspaces/{workspace_id}	Real-time events (messages, presence, typing)

Request Expectations

- Register requires `username`, `email`, `password`, `display_name`
- Login uses the OAuth2 password form (`username`, `password`)
- Message creation requires `body` and optional `parent_id` for a threaded reply
- Channel creation requires `name`, `is_private`, and optional `topic`
- Invite creation requires `email` and `target` `role`
- WebSocket authentication must pass the access token via a query parameter or the first message, validated before the connection is accepted

Response Expectations

- Login returns `access_token`, `refresh_token`, and `token_type`
- Message responses include generated `id`, `sender`, `created_at`, `edited_at`, `reaction` summary, and thread info; never expose internal soft-delete flags
- History responses include `items`, `next_cursor`, and `has_more`
- User responses never include the password hash
- WebSocket event payloads use a typed envelope such as `{ "type": "message.created", "data": { ... } }`

Validation Requirements

- `username`: 3-30 characters, unique
- `email`: valid format, unique

- `password`: 8-72 characters (respect bcrypt's limit)
- `Workspace name`: 2-80 characters; `slug` unique
- `Channel name`: 2-80 characters, unique within a workspace
- `Message body`: 1-4000 characters
- `emoji`: constrained to a short allowed set or a simple pattern
- `Role`: one of `owner`, `admin`, `member`
- `Pagination limit`: 1-100; cursor must be an opaque, validated token

Business Rules

- A newly created workspace has its creator as the sole `owner`
- Only `owner` or `admin` may invite, remove members, or change roles; a workspace must always retain at least one `owner`
- Private channels are visible and postable only to their members; public channels are visible to all workspace members
- Only the message author may edit or delete their message; admins may delete any message in their workspace
- Editing a message sets `edited_at` and must broadcast an update event
- A reaction is unique per user, message, and emoji
- Sending a message must broadcast a real-time event to all connected channel members
- Rate limiting must throttle message sending per user and return 429 when exceeded
- Invites expire after a fixed period and cannot be reused

Edge Cases

- Logging in with a wrong password returns 401 with a generic message
- Accessing a workspace or channel the caller does not belong to returns 404 (do not leak existence)
- Posting to a private channel without membership returns 403
- Removing the last owner returns 409
- Reacting twice with the same emoji is idempotent
- A WebSocket connection with a missing or invalid token is rejected before acceptance
- Paginating with a tampered cursor returns 400
- Editing another user's message returns 403

Suggested Database Schema

- `users` (`id`, `username` unique, `email` unique, `hashed_password`, `display_name`, `created_at`)
- `workspaces` (`id`, `name`, `slug` unique, `owner_id` -> `users.id`, `created_at`)
- `workspace_members` (`id`, `workspace_id` -> `workspaces.id`, `user_id` -> `users.id`, `role`, `joined_at`, `unique(workspace_id, user_id)`)
- `channels` (`id`, `workspace_id` -> `workspaces.id`, `name`, `is_private`, `topic`, `created_by` -> `users.id`, `created_at`)

- `channel_members` (`channel_id` -> `channels.id`, `user_id` -> `users.id`, `unique(channel_id, user_id)`)
- `messages` (`id`, `channel_id` -> `channels.id`, `sender_id` -> `users.id`, `body`, `parent_id` -> `messages.id` nullable, `created_at`, `edited_at`, `is_deleted`)
- `reactions` (`id`, `message_id` -> `messages.id`, `user_id` -> `users.id`, `emoji`, `unique(message_id, user_id, emoji)`)
- `invites` (`id`, `workspace_id` -> `workspaces.id`, `email`, `token` unique, `role`, `invited_by` -> `users.id`, `status`, `expires_at`)

Relationships to model:

- Workspace to Member to User: many-to-many via `workspace_members` (association object with a role)
- Workspace to Channel: one-to-many
- Channel to Message: one-to-many
- Message self-referential for threads (parent to replies)
- Message to Reaction: one-to-many

Expected Folder Structure

```
team_chat_api/
  main.py
  database.py
  models.py
  schemas.py
  security.py
  dependencies.py
  exceptions.py
  realtime/
    manager.py
  background/
    notifications.py
  routers/
    auth.py
    workspaces.py
    channels.py
    messages.py
    reactions.py
    ws.py
  alembic.ini
  alembic/
    env.py
    versions/
  seed.py
```

Deliverables

- Runnable, database-backed FastAPI app with JWT auth and real-time messaging
- Alembic migrations that build the full schema

- A WebSocket connection manager with presence and broadcasting
- Seed data with several users, at least 2 workspaces, channels, and threaded messages
- README with the auth flow, the WebSocket protocol, and example requests
- At least 15 documented manual test cases including auth, RBAC, real-time, and pagination

Evaluation Criteria

- Correct, secure JWT auth and workspace/channel-scoped authorization
- Correct WebSocket lifecycle: authentication, broadcasting, disconnect cleanup
- Correct cursor pagination and message search
- Sensible rate limiting and caching with invalidation
- Clean modular structure and consistent error handling

Bonus Challenges

- Add typing indicators and online-presence tracking broadcast over WebSockets
- Add unread-message counts per channel using a background task and cached counters
- Add message search with filtering by sender and date range
- Add an SSE fallback stream for clients that cannot use WebSockets

Project 2 - Live Auction and Bidding Platform API

Difficulty Level

Very Advanced

Estimated Completion Time

28-36 hours

Project Overview

Build the backend for a live auction platform where sellers list items, bidders compete in real time, and auctions close automatically at their end time. This project centers on concurrency-safe bidding (atomic transactions preventing race conditions on the highest bid), real-time price feeds, scheduled auction closing via background tasks, and cached hot-auction data under heavy read load.

Problem Statement

An auction marketplace needs a backend to:

- Authenticate sellers, bidders, and admins
- Let sellers create and schedule auctions
- Let bidders place bids and set automatic proxy bids
- Broadcast price changes to everyone watching an auction in real time
- Close auctions exactly at their end time and determine winners
- Notify winners and outbid users
- Serve popular auctions quickly under load

Functional Requirements

- Register and authenticate users with roles
- Create, schedule, start, and cancel auctions
- Place manual bids and configure proxy (maximum) auto-bids
- Maintain and broadcast the current highest bid in real time
- Watchlist auctions and receive notifications
- Automatically close auctions at their end time and assign a winner
- List and page through bid history

Non-Functional Requirements

- Concurrency-safe bidding using database transactions to prevent lost updates on the highest bid
- Real-time price updates over WebSockets or SSE
- Scheduled auction closing via background tasks
- Caching of hot auction state with invalidation on each accepted bid
- Rate limiting to prevent bid spamming

API Requirements

- Use routers such as `auth`, `auctions`, `bids`, `watchlist`, `notifications`, and a real-time module
- Use JWT auth with roles `bidder`, `seller`, `admin`
- Place bids inside a single database transaction that re-checks the current highest bid before accepting
- Broadcast an accepted bid to all connected watchers
- Cache each live auction's current price and top bidders, invalidating on a new accepted bid

Required API Endpoints

Method	Path	Purpose
POST	<code>/auth/register</code>	Create a user
POST	<code>/auth/login</code>	Login, returns tokens

POST	/auth/refresh	Refresh access token
POST	/auctions	Create an auction (seller)
GET	/auctions	List/search/filter auctions
GET	/auctions/{auction_id}	Get one auction with current price
POST	/auctions/{auction_id}/schedule	Schedule an auction to go live
POST	/auctions/{auction_id}/cancel	Cancel an auction
POST	/auctions/{auction_id}/bids	Place a bid
GET	/auctions/{auction_id}/bids	Cursor-paginated bid history
POST	/auctions/{auction_id}/auto-bid	Set a proxy max bid
POST	/auctions/{auction_id}/watch	Add to watchlist
DELETE	/auctions/{auction_id}/watch	Remove from watchlist
GET	/notifications	List the caller's notifications
GET	/auctions/{auction_id}/price-stream	SSE live price feed
WS	/ws/auctions/{auction_id}	Real-time bid events

Request Expectations

- Auction creation requires `title`, `description`, `category`, `starting_price`, optional `reserve_price`, `starts_at`, and `ends_at`
- Bid placement requires `amount`
- Auto-bid requires `max_amount`
- All write endpoints require a valid access token

Response Expectations

- Auction responses include `status`, `current_price`, `highest_bidder` (masked as needed), `bid_count`, and time fields
- A successful bid returns the new `current_price` and the bid record; a losing/too-low bid returns a clear 409
- Bid history responses are cursor-paginated
- Real-time events use a typed envelope such as `{ "type": "bid.accepted", "data": { ... } }`

Validation Requirements

- `starting_price` and `reserve_price`: greater than zero; reserve, if present, must be greater than or equal to starting price
- `ends_at` must be after `starts_at` and in the future at creation
- `Bid amount`: greater than zero
- `category`: one of a controlled set
- `Role`: one of `bidder`, `seller`, `admin`
- `Pagination limit`: 1-100

Business Rules

- Only a `seller` may create auctions; only the owning seller or an `admin` may cancel one
- Auctions move through `draft`, `scheduled`, `live`, `ended`, and `cancelled`; only valid transitions are allowed
- A bid is accepted only if the auction is `live` and the amount exceeds the current highest bid by at least a minimum increment
- Bid acceptance must be atomic: two simultaneous bids must not both become the highest bid
- A seller cannot bid on their own auction
- Placing a higher bid must mark the previous highest bidder as `outbid` and notify them
- Proxy auto-bids automatically raise a bidder's bid up to their maximum when they are `outbid`
- An auction with bids below its reserve price ends without a winner
- Closing an auction is triggered by a background task at `ends_at`, sets the winner, and notifies participants
- Each accepted bid invalidates the cached auction state

Edge Cases

- Bidding on a non-live auction returns 409
- Bidding below the current highest plus increment returns 409
- A seller bidding on their own auction returns 403
- Cancelling an auction that already has bids follows a defined policy (for example, allowed only before it goes live)
- Two concurrent equal bids must resolve to exactly one winner
- Bidding after the end time returns 409 even if closing has not yet been processed
- Setting an auto-bid below the current price returns 409

Suggested Database Schema

- `users` (`id`, `username` unique, `email` unique, `hashed_password`, `role`, `created_at`)
- `auctions` (`id`, `seller_id` -> `users.id`, `title`, `description`, `category`, `starting_price`, `reserve_price`, `current_price`, `status`, `starts_at`, `ends_at`, `winner_id` -> `users.id` nullable, `created_at`)
- `bids` (`id`, `auction_id` -> `auctions.id`, `bidder_id` -> `users.id`, `amount`, `placed_at`)
- `auto_bids` (`id`, `auction_id` -> `auctions.id`, `bidder_id` -> `users.id`, `max_amount`, `unique(auction_id, bidder_id)`)

- `watchlist` (`user_id` -> `users.id`, `auction_id` -> `auctions.id`, `unique(user_id, auction_id)`)
- `notifications` (`id`, `user_id` -> `users.id`, `type`, `payload`, `is_read`, `created_at`)

Relationships to model:

- Seller to Auction: one-to-many
- Auction to Bid: one-to-many
- User to Auction via `watchlist`: many-to-many
- Auction to AutoBid: one-to-many

Expected Folder Structure

```
live_auction_api/  
  main.py  
  database.py  
  models.py  
  schemas.py  
  security.py  
  dependencies.py  
  exceptions.py  
  realtime/  
    manager.py  
  background/  
    closing.py  
  services/  
    bidding.py  
  routers/  
    auth.py  
    auctions.py  
    bids.py  
    watchlist.py  
    notifications.py  
    realtime.py  
  alembic.ini  
  alembic/  
    env.py  
    versions/  
  seed.py
```

Deliverables

- Runnable async FastAPI app with JWT auth and real-time bidding
- Alembic migrations building the full schema
- Atomic bid placement demonstrated in the README (including the concurrency argument)
- Background-task auction closing that assigns winners
- Seed data with sellers, bidders, several auctions across statuses, and bids
- README covering the auction lifecycle, the bidding transaction, and the real-time protocol

- At least 15 documented manual test cases including concurrency, RBAC, and lifecycle transitions

Evaluation Criteria

- Correct, atomic, concurrency-safe bid handling
- Correct auction lifecycle and background-task closing
- Correct real-time price broadcasting
- Effective caching and rate limiting on the hot bidding path
- Clean separation of bidding logic, real-time, and background concerns

Bonus Challenges

- Implement proxy auto-bidding fully, including cascading auto-bid wars resolved atomically
- Add anti-sniping: extend an auction's end time by a short interval if a bid lands in the final seconds
- Add a cached, real-time leaderboard of top auctions by bid activity
- Add an SSE endpoint that streams a personal feed of the caller's outbid notifications

Project 3 - IoT Smart-Building Telemetry and Device Management API

Difficulty Level

Expert

Estimated Completion Time

32-42 hours

Project Overview

Build a multi-tenant backend for smart-building monitoring, where gateways and devices stream sensor readings, operators watch live dashboards, and the system evaluates alert rules and rolls up aggregates. This project combines dual authentication (API keys for devices, JWT for humans), high-volume asynchronous ingestion, per-device rate limiting, real-time dashboards, background aggregation and alerting, and time-series querying with cursor pagination.

Problem Statement

A facilities platform needs a backend to:

- Isolate data per organization (multi-tenant)
- Authenticate human users with JWT and devices with API keys
- Ingest sensor readings at high volume
- Serve live readings to dashboards in real time
- Evaluate alert rules and raise alerts
- Aggregate readings into hourly and daily rollups
- Query historical time-series efficiently

Functional Requirements

- Register organizations and users; assign roles
- Provision gateways and devices; issue and rotate device API keys
- Ingest readings from devices, authenticated by API key
- Stream live readings to operators over SSE or WebSockets
- Define alert rules and evaluate them as readings arrive
- Acknowledge and resolve alerts
- Produce and serve hourly/daily aggregates
- Query readings with time-range filtering, sorting, and cursor pagination

Non-Functional Requirements

- Strict multi-tenant isolation enforced on every query
- Asynchronous ingestion that does not block the event loop
- Per-device / per-API-key rate limiting on ingestion
- Background tasks for aggregation rollups and alert evaluation
- Caching of the latest reading per sensor and of computed aggregates
- Cursor-based pagination for time-series data

API Requirements

- Use routers such as `auth`, `organizations`, `gateways`, `devices`, `ingestion`, `readings`, `alerts`, and a `real-time` module
- Authenticate humans with JWT and devices with an API key header; never mix the two credential types on one endpoint
- Enforce organization scoping through a dependency on every data route
- Ingest readings through an async endpoint, rate limited per device
- Broadcast live readings to subscribed operators through a connection manager
- Cache the latest value per sensor and invalidate on new readings

Required API Endpoints

Method	Path	Purpose
POST	/auth/register	Create a user in an organization
POST	/auth/login	Login, returns tokens
POST	/auth/refresh	Refresh access token
POST	/organizations	Create an organization (admin)
POST	/gateways	Register a gateway
GET	/gateways	List gateways in the org
POST	/devices	Provision a device (returns an API key once)
POST	/devices/{device_id}/rotate-key	Rotate the device API key
GET	/devices	List/filter devices
POST	/devices/{device_id}/sensors	Add a sensor to a device
POST	/ingest/readings	Ingest readings (device API key)
GET	/sensors/{sensor_id}/readings	Query time-series (filter, sort, paginate)
GET	/sensors/{sensor_id}/aggregates	Get hourly/daily rollups
POST	/alert-rules	Create an alert rule
GET	/alerts	List/filter alerts
POST	/alerts/{alert_id}/acknowledge	Acknowledge an alert
GET	/sensors/{sensor_id}/live	SSE live reading feed
WS	/ws/organizations/{org_id}/dashboard	Real-time dashboard stream

Request Expectations

- Device provisioning requires `gateway_id`, `name`, and `type`
- Reading ingestion requires the device API key header and a payload of one or more { `sensor_id`, `value`, `recorded_at` } items
- Alert rule creation requires `metric`, `operator`, `threshold`, `severity`, and an optional sensor scope
- Reading queries accept `start`, `end`, `metric`, `limit`, and `cursor`

Response Expectations

- Device provisioning returns the raw API key exactly once; subsequent responses expose only a masked identifier

- Reading query responses are cursor-paginated with `items`, `next_cursor`, and `has_more`
- Aggregate responses include bucket start, min, max, avg, and count
- Alerts include the triggering value, rule, severity, and status
- SSE and WebSocket payloads use a typed envelope

Validation Requirements

- `metric`: one of a controlled set (for example `temperature`, `humidity`, `co2`, `power`)
- `operator`: one of `gt`, `gte`, `lt`, `lte`, `eq`
- `value`: numeric; `recorded_at`: a valid timestamp not unreasonably in the future
- `severity`: one of `info`, `warning`, `critical`
- Ingestion batch size: bounded (for example at most 500 readings per request)
- Time-range queries: `start` must not be after `end`
- Pagination `limit`: 1-1000 for time-series reads

Business Rules

- Every device, sensor, reading, alert, and query is scoped to exactly one organization; cross-tenant access returns not found
- Reading ingestion authenticates with the device's API key and updates the device `last_seen_at`
- An API key is stored hashed; the raw key is shown only at creation and on rotation
- Rotating a key immediately invalidates the previous key
- New readings must update the cached latest value and broadcast to subscribed dashboards
- Alert rules are evaluated as matching readings arrive; a breach raises an alert and notifies operators
- Aggregation rollups are produced by a background task and must be idempotent for a given bucket
- Ingestion is rate limited per device and returns 429 when exceeded

Edge Cases

- Ingesting with an invalid or rotated API key returns 401
- Ingesting for a sensor in another organization returns 404
- A future-dated or malformed reading is rejected
- Querying a time range with `start` after `end` returns 422
- Exceeding the ingestion batch limit returns 422
- Acknowledging an already-resolved alert returns 409
- A JWT user attempting device-only ingestion is rejected

Suggested Database Schema

- `organizations` (`id`, `name`, `slug` unique, `created_at`)
- `users` (`id`, `organization_id` -> `organizations.id`, `username` unique, `email`, `hashed_password`, `role`,

created_at)

- gateways (id, organization_id -> organizations.id, name, location, status)
- devices (id, organization_id -> organizations.id, gateway_id -> gateways.id, name, type, api_key_hash, status, last_seen_at)
- sensors (id, device_id -> devices.id, metric, unit, unique(device_id, metric))
- readings (id, sensor_id -> sensors.id, value, recorded_at)
- alert_rules (id, organization_id -> organizations.id, sensor_id -> sensors.id nullable, metric, operator, threshold, severity, is_active)
- alerts (id, rule_id -> alert_rules.id, sensor_id -> sensors.id, value, triggered_at, status, acknowledged_by -> users.id nullable)
- aggregates (id, sensor_id -> sensors.id, period, bucket_start, min_value, max_value, avg_value, count, unique(sensor_id, period, bucket_start))

Relationships to model:

- Organization to User, Gateway, Device: one-to-many
- Gateway to Device: one-to-many
- Device to Sensor: one-to-many
- Sensor to Reading and to Aggregate: one-to-many
- Alert rule to Alert: one-to-many

Expected Folder Structure

```
iot_telemetry_api/  
  main.py  
  database.py  
  models.py  
  schemas.py  
  security.py  
  dependencies.py  
  exceptions.py  
  realtime/  
    manager.py  
  background/  
    aggregation.py  
    alerting.py  
  routers/  
    auth.py  
    organizations.py  
    gateways.py  
    devices.py  
    ingestion.py  
    readings.py  
    alerts.py  
    realtime.py  
  alembic.ini  
  alembic/  
    env.py
```

```
versions/  
seed.py
```

Deliverables

- Runnable async FastAPI app with dual auth (JWT + API keys) and multi-tenant isolation
- Alembic migrations building the full schema
- Async ingestion with per-device rate limiting
- Background aggregation and alert evaluation
- SSE or WebSocket live dashboard streaming
- Seed data with organizations, gateways, devices, sensors, and historical readings
- README covering tenancy, device auth, ingestion, and the real-time dashboard
- At least 16 documented manual test cases including tenant isolation, dual auth, and time-series pagination

Evaluation Criteria

- Correct multi-tenant isolation and dual authentication
- Non-blocking, rate-limited ingestion
- Correct background aggregation and alerting
- Effective caching of latest values and aggregates
- Correct time-series filtering and cursor pagination

Bonus Challenges

- Store raw readings in MongoDB while keeping devices and rules in SQL, and build queries against the document store
- Add a Redis-backed sliding-window rate limiter per device shared across workers
- Add downsampling that serves different aggregate resolutions based on the requested time range
- Add an operator dashboard WebSocket that streams only alerts above a chosen severity

Project 4 - On-Demand Delivery Dispatch and Tracking Platform API

Difficulty Level

Expert (Capstone)

Estimated Completion Time

40-52 hours

Project Overview

Build the capstone: the backend for an on-demand delivery platform where customers place orders, the system dispatches them to nearby couriers, couriers stream live location, and customers track their delivery in real time. This project integrates nearly every Phase 1-4 concept: JWT auth with three roles, atomic dispatch (only one courier can claim an order), background matching and offer expiry, WebSocket location ingestion, SSE customer tracking, caching of available couriers, rate limiting, and cursor-paginated history.

Problem Statement

A logistics platform needs a backend to:

- Authenticate customers, couriers, and admins
- Accept delivery orders from customers
- Dispatch each order to nearby available couriers as time-limited offers
- Let exactly one courier accept an order, safely under concurrency
- Track courier location and order status in real time
- Let customers follow their delivery live
- Record ratings and produce operational history

Functional Requirements

- Register and authenticate users with roles (customer, courier, admin)
- Manage courier availability and live location
- Create orders and move them through a delivery lifecycle
- Dispatch orders as offers to candidate couriers and expire unaccepted offers
- Accept an offer atomically so only one courier is assigned
- Stream courier location and order status to the right audience in real time
- Rate completed deliveries and list history

Non-Functional Requirements

- Async database access throughout
- Atomic offer acceptance preventing double-assignment under concurrency
- Background tasks for dispatch matching, offer expiry, and receipt generation
- WebSocket ingestion of courier location and SSE tracking for customers
- Caching of available couriers and current order status
- Rate limiting on location pings and order creation

API Requirements

- Use routers such as `auth`, `couriers`, `orders`, `dispatch`, `tracking`, `ratings`, and `real-time` modules
- Use JWT auth with roles `customer`, `courier`, `admin`; scope every action to the caller's role and ownership
- Accept an offer inside a single database transaction that assigns the order only if still unassigned
- Ingest courier location over a WebSocket and broadcast relevant updates
- Stream order tracking to the owning customer over SSE
- Cache the set of available couriers and invalidate it on status changes

Required API Endpoints

Method	Path	Purpose
POST	<code>/auth/register</code>	Create a customer or courier
POST	<code>/auth/login</code>	Login, returns tokens
POST	<code>/auth/refresh</code>	Refresh access token
PATCH	<code>/couriers/me/status</code>	Set courier availability
POST	<code>/orders</code>	Create a delivery order
GET	<code>/orders</code>	List the caller's orders (cursor paginated)
GET	<code>/orders/{order_id}</code>	Get one order
POST	<code>/orders/{order_id}/cancel</code>	Cancel an order when allowed
GET	<code>/dispatch/offers</code>	Courier lists their pending offers
POST	<code>/dispatch/offers/{offer_id}/accept</code>	Accept an offer (atomic)
POST	<code>/dispatch/offers/{offer_id}/decline</code>	Decline an offer
POST	<code>/orders/{order_id}/pickup</code>	Courier marks picked up
POST	<code>/orders/{order_id}/deliver</code>	Courier marks delivered
POST	<code>/orders/{order_id}/ratings</code>	Rate a completed delivery
GET	<code>/admin/orders</code>	Admin operational list with filters
WS	<code>/ws/couriers/location</code>	Courier streams location
GET	<code>/orders/{order_id}/track</code>	SSE live tracking for the customer

Request Expectations

- Order creation requires `merchant_id` or pickup details, `dropoff_address`, and item summary
- Courier status update requires a `status` of `offline`, `available`, or `on_trip`
- Location pings require `lat`, `lng`, and a timestamp, sent over the WebSocket
- Offer acceptance and lifecycle transitions require the courier's access token

Response Expectations

- Order responses include `status`, assigned courier (when present), timestamps, and total
- Offer acceptance returns the assigned order or a 409 if the offer is already taken or expired
- History responses are cursor-paginated
- Tracking SSE events include courier position and order status transitions in a typed envelope
- Customer and courier responses hide each other's sensitive contact fields beyond what a delivery requires

Validation Requirements

- `lat`: -90 to 90; `lng`: -180 to 180
- Order addresses: non-empty, bounded length
- `status` transitions constrained to the defined lifecycle
- Rating `stars`: 1-5; `comment`: bounded length
- Role: one of `customer`, `courier`, `admin`
- Pagination `limit`: 1-100

Business Rules

- Orders move through `created`, `dispatching`, `assigned`, `picked_up`, `delivered`, and `cancelled`; only valid transitions are allowed
- On creation, a background task dispatches the order as time-limited offers to nearby available couriers
- Exactly one courier can accept an order; acceptance must be atomic so concurrent accepts cannot double-assign
- Accepting an offer marks the courier `on_trip`, sets the order `assigned`, and expires the order's other offers
- Offers expire after a fixed interval; expired offers are re-dispatched by a background task
- Only the assigned courier may mark pickup and delivery, in order
- Only the owning customer may cancel, and only before pickup
- Delivery completion triggers a background receipt and makes the courier `available` again
- Ratings are allowed only after delivery and only by the two parties involved
- Available-courier lookups are cached and invalidated when courier status changes

Edge Cases

- Two couriers accepting the same offer simultaneously result in exactly one assignment; the other receives 409
- Accepting an expired offer returns 409
- A non-assigned courier marking pickup or delivery returns 403
- Cancelling after pickup returns 409
- A location ping over an unauthenticated WebSocket is rejected
- Rating an order that is not delivered returns 409
- A customer accessing another customer's order returns 404

Suggested Database Schema

- `users` (id, username unique, email unique, hashed_password, role, created_at)
- `couriers` (id, user_id -> users.id unique, vehicle_type, status, current_lat, current_lng, rating_avg, updated_at)
- `merchants` (id, name, address, location_lat, location_lng)
- `orders` (id, customer_id -> users.id, merchant_id -> merchants.id nullable, pickup_address, dropoff_address, status, total_amount, assigned_courier_id -> couriers.id nullable, created_at)
- `dispatch_offers` (id, order_id -> orders.id, courier_id -> couriers.id, status, offered_at, expires_at, unique(order_id, courier_id))
- `location_pings` (id, courier_id -> couriers.id, lat, lng, recorded_at)
- `ratings` (id, order_id -> orders.id, rater_id -> users.id, ratee_role, stars, comment, created_at)
- `notifications` (id, user_id -> users.id, type, payload, is_read, created_at)

Relationships to model:

- User to Courier: one-to-one
- Customer to Order: one-to-many
- Order to DispatchOffer: one-to-many
- Courier to Order (assignment): one-to-many
- Order to Rating: one-to-many

Expected Folder Structure

```
delivery_dispatch_api/
  main.py
  database.py
  models.py
  schemas.py
  security.py
  dependencies.py
  exceptions.py
  realtime/
    location_manager.py
    tracking.py
  background/
```

```
dispatch.py
offers.py
receipts.py
services/
  assignment.py
routers/
  auth.py
  couriers.py
  orders.py
  dispatch.py
  ratings.py
  admin.py
  realtime.py
alembic.ini
alembic/
  env.py
  versions/
seed.py
```

Deliverables

- Runnable async FastAPI app integrating auth, real-time, background tasks, caching, and rate limiting
- Alembic migrations building the full schema
- Atomic offer acceptance demonstrated in the README, including the concurrency argument
- Background dispatch matching, offer expiry, and receipt generation
- WebSocket location ingestion and SSE customer tracking
- Seed data with customers, couriers, merchants, and orders across the lifecycle
- README covering the dispatch flow, the assignment transaction, and the real-time protocols
- At least 18 documented manual test cases including concurrency, RBAC, lifecycle, and real-time

Evaluation Criteria

- Correct, atomic, concurrency-safe dispatch and assignment
- Correct role-based authorization and ownership checks throughout
- Correct real-time location ingestion and customer tracking
- Correct background dispatch, expiry, and receipt handling
- Effective caching, rate limiting, and cursor pagination
- Clean, modular, maintainable architecture across many concerns

Bonus Challenges

- Add nearest-courier matching using stored coordinates and a bounded search radius
- Add an admin analytics surface via GraphQL (Strawberry) for orders, couriers, and delivery times
- Add internationalized customer notifications negotiated from `Accept-Language`

- Add a Redis-backed shared cache and rate limiter so the system behaves correctly across multiple workers

Final Submission Checklist

For each project, submit:

- Full project folder with a modular structure
- `main.py`, `database.py`, `models.py`, `schemas.py`, `security.py`
- Router modules, dependency module, and real-time manager module(s)
- Background-task module(s) and any service modules
- Exception-handling module if used
- `alembic/` directory with real, ordered migration scripts
- A seed script or seed data
- README with setup steps, the auth flow, the real-time protocol, and example requests

Before submitting, verify:

- The schema is created entirely by Alembic migrations (`alembic upgrade head` on an empty database works)
- The app starts with `uvicorn main:app --reload` and `/docs` shows all routes
- Registration, login, token refresh, and a protected route all work
- Role-based authorization blocks unauthorized actions with `403` and unknown resources with `404`
- Real-time endpoints authenticate before accepting a connection and clean up on disconnect
- Concurrency-sensitive operations (bids, dispatch acceptance) are atomic and cannot double-apply
- Background tasks run after the response and complete their work
- Rate limiting returns `429` and caching invalidates correctly on writes
- Cursor pagination is stable and `limit` is bounded
- No Phase 5+ topic (automated testing, structured logging, deployment) was used as a requirement

Difficulty Progression

Project	Difficulty	Main Focus
Real-Time Team Chat and Collaboration API	Advanced	JWT auth, workspace RBAC, WebSocket broadcasting, cursor history
Live Auction and Bidding Platform API	Very Advanced	Atomic bidding, real-time price feeds, scheduled closing, caching
IoT Smart-Building Telemetry and Device Management API	Expert	Multi-tenant, dual auth, async ingestion, background aggregation, time-series

On-Demand Delivery Dispatch and Tracking Platform API	Expert (Capstone)	Atomic dispatch, real-time location, background matching, full integration
---	-------------------	--

Complete the projects in order. Each assumes fluency with everything before it. By the end, you will have built four real-time, secure, database-backed backends that together exercise every concept from Lessons 1-39 - a genuine capstone for Phase 1 through Phase 4.

--	--	--

--	--	--

--	--	--

--	--	--